SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING **NAME : E.LOHITHA**

COURSE CODE: CSA4303 **REGNO:192411103**

MARKS:90

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

1: **HTTP Response Analysis**

A web server returns the following HTTP response header:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150 Connection:

keep-alive

Questions:

1. Identify any incorrect or invalid fields in the header.

2. Analyze the issue with Content-Length and explain its impact.

3. Determine whether the status code matches the response correctness.

4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

<**html>**

<head>

<title>TestPage</title>

</head>

<body>

** **

<p>HelloWorld

** **

</body>

</html>

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	**URL Example**	**Data Visibility**	**Security Level**
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication. 4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

<!DOCTYPE html>

<html>

<head>

<title>Sample</title>

</head>

<body>

<h1>Welcome

<p>This is a paragraph

<img src="image.jpg">

</body>

</html>

Questions:

1. Identify missing closing tags.
2. Analyze issues with the <img> tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned 6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I __ E.LOHITHA___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	**Weigh tage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**MARKS**
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	22.5
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis wit h some justification	Basic analysis with weak reasoning	No meaningful analysis provided	22.5
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts wi th minor errors	Limited application; requires guidance	Unable to apply concepts to data	18
Conclusion	20%	Draws clear, meaningful conclusi ons with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	17.5

Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	7.5

ANSWERS

1.HTTP Response Analysis

1. **A web server returns the following HTTP response header:**
2. **HTTP/1.1 200 OK**
3. **Date: Mon, 01 Apr 2026 10:00:00 GMT**
4. **Content-Type: text/html**
5. **Content-Length: -150**
6. **Connection: keep-alive**

Questions:

1.1 Identify any incorrect or invalid fields in the header.

The given HTTP response header contains the following errors:

1. The Date field is incorrect because 1 April 2026 falls on Wednesday, not Monday.
2. The Content-Length field is invalid because its value is -150. Content-Length must be a non-negative integer representing the response body size in bytes.
3. The status code 200 OK indicates success, but the response contains an invalid header value.

1.2 Analyze the issue with Content-Length and explain its impact.

The Content-Length header specifies the size of the HTTP response body in bytes. The value -150 is invalid because a content length cannot be negative. This may cause the client to reject the response or handle it incorrectly. It can also create problems with message framing and connection handling.

1.3 Determine whether the status code matches the response correctness.

The status code 200 OK indicates that the request was successfully processed. However, it does not guarantee that all response headers are valid. Therefore, the response has a success status code but contains malformed metadata that should be corrected.

1.4 Suggest corrections for the identified errors.

The corrected HTTP response header is:

HTTP/1.1 200 OK

Date: Wed, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

The Content-Length value must match the actual size of the response body in bytes. The value 150 is correct only if the body contains exactly 150 bytes. The Date field has also been corrected. The Connection field is valid for HTTP/1.1.

2. DOM Structure Analysis

Consider the following DOM tree representation:

<**html>**

<head>

<title>TestPage</title>

</head>

<body>

** **

** HelloWorld**

** **

</body>

</html>

Questions:

2.1 Identify missing or improperly closed elements.

The given DOM structure contains a missing closing tag for the paragraph element . The opening tag is not explicitly closed using before the closing tag. The other elements have their corresponding closing tags.

2.2 Analyze how the browser will interpret this structure.

The browser automatically corrects many HTML markup errors during parsing. When the browser encounters the closing tag, it implicitly closes the open paragraph element. Therefore, the browser generally creates a paragraph element containing the text "HelloWorld" inside the div element.

2.3 Determine the impact on rendering and layout.

The browser will generally display the text "HelloWorld" correctly. However, missing closing tags can cause unexpected nesting, spacing, and layout problems in more complex documents. Such errors can also reduce code readability and make debugging difficult.

2.4 Suggest corrections to make the DOM valid.

The corrected HTML code is:

```
<!DOCTYPE html>
<html>
<head>
  <title>TestPage</title>
</head>
<body>
  <div>
    <p>HelloWorld</p>
  </div>
</body>
</html>
```

This corrected version contains properly nested and explicitly closed elements, making the document easier to read, maintain, and validate.

3. GET vs POST in a Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

3.1 Interpret the differences in data transmission between GET and POST.

GET and POST are HTTP methods used to communicate between a browser and a web server.

GET generally sends data through URL query parameters. Therefore, the data may appear in the browser address bar, browser history, and server logs. GET is mainly used to retrieve information from a server.

POST generally sends data in the request body rather than the URL. It is commonly used to submit forms, send login credentials, and process data on the server.

However, POST does not automatically encrypt data. HTTPS is required to protect information during transmission.

3.2 Analyze which method is more suitable for login and why.

POST is more suitable for login authentication because it sends usernames and passwords in the request body instead of placing them directly in the URL. This reduces the risk of exposing credentials through browser history and copied links.

For secure authentication, POST must be used over HTTPS. The server must also validate credentials and store passwords securely.

3.3 Identify security risks in using GET for authentication.

The main security risks of using GET for authentication are:

1. Passwords may appear in the browser address bar and browsing history.
2. URLs containing credentials may be stored in server logs and proxy logs.
3. Users may accidentally share URLs containing sensitive information.
4. Credentials may be exposed through monitoring systems or other URL-handling mechanisms.
5. Attackers may misuse exposed credentials to gain unauthorized access.

Therefore, GET should not be used to transmit passwords or other sensitive authentication information.

3.4 Suggest best practices for secure login implementation.

The following best practices should be followed:

1. Use POST for submitting login credentials.
2. Use HTTPS to encrypt communication between the browser and server.
3. Store passwords using secure password-hashing algorithms such as Argon2id or bcrypt.
4. Never store passwords in plain text.
5. Use secure session cookies with the HttpOnly, Secure, and appropriate SameSite attributes.
6. Implement rate limiting to prevent brute-force attacks.

7. Validate user input on the server side.
8. Use multi-factor authentication when appropriate.
9. Provide appropriate error messages without revealing unnecessary account information.
10. Protect authenticated sessions and expire them when necessary.

In conclusion, POST over HTTPS, combined with secure password storage and session management, provides a safer login system.

4.HTML Code Inspection

Analyze the following HTML snippet:

<!DOCTYPE html>

<html>

<head>

<title>Sample</title>

</head>

<body>

<h1>Welcome

** This is a paragraph**

<img src="image.jpg">

</body>

</html>

Questions:

4.1 Identify missing closing tags.

The given HTML code has missing closing tags for the <h1> and elements. The heading and paragraph elements should be explicitly closed using </h1> and .

The <img> element does not require a closing tag in HTML because it is a void element.

4.2 Analyze issues with the <img> tag.

The given image tag is:

<img src="image.jpg">

The main issue is that the image tag does not contain an alt attribute. Alternative text helps users who rely on screen readers understand the image. It is also useful when the image cannot be loaded.

The image may not appear if the file image.jpg does not exist at the specified path. Adding suitable width and height attributes can also help reduce layout shifts.

4.3 Determine how browsers will handle these errors.

Browsers attempt to recover from common HTML errors automatically. The browser will generally close the heading when it encounters the paragraph element and close the paragraph when it reaches the end of the body.

The image will be displayed if the specified file exists and loads successfully. Otherwise, the browser may display a broken-image indicator. Missing alternative text reduces accessibility.

4.4 Suggest a corrected version of the code.

The corrected HTML code is:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph.</p>
  <img src="image.jpg" alt="Sample image" width="400" height="300">
</body>
</html>
```

This version includes proper closing tags, character encoding, viewport settings, and alternative text for the image. The image dimensions should be adjusted according to the actual image.

5.Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser

2. DNS lookup occurs

3. HTTP request is sent

4. Server processes request

5. Response is returned 6. Browser renders page

Questions:

5.1 Interpret each step in the communication flow.

Step 1: User enters a URL in the browser

The user enters a website address in the browser. The browser identifies the protocol, domain name, and requested resource.

Step 2: DNS lookup occurs

The Domain Name System (DNS) resolves the domain name into an IP address. The browser or operating system may use a cached address to avoid a new lookup.

Step 3: HTTP request is sent

The browser establishes a connection with the server. If HTTPS is used, a TLS handshake is performed to establish secure communication. The browser then sends an HTTP request containing the method, requested path, headers, and any required body.

Step 4: Server processes the request

The web server receives the request and processes it. It may execute application code, access a database, retrieve files, and prepare the appropriate response.

Step 5: Response is returned

The server sends an HTTP response containing a status code, response headers, and, when applicable, a response body containing HTML, images, JSON, or other resources.

Step 6: Browser renders the page

The browser parses the HTML, constructs the Document Object Model (DOM), processes CSS, executes JavaScript, and displays the webpage. Additional requests may be required to load images, stylesheets, and scripts.

5.2 Identify where delays can occur.

Delays can occur at different stages of browser-server communication:

1. DNS lookup may take time if the DNS cache is empty or the DNS server is slow.
2. Connection establishment and TLS handshakes may increase connection time.
3. Network congestion and packet loss may delay request and response transmission.
4. Server processing may be slow because of complex application logic or database queries.

5. Large files and unoptimized images may increase download time.
6. Complex HTML, CSS, and JavaScript may delay browser rendering.

5.3 Analyze the role of DNS in the process.

DNS stands for Domain Name System. It translates human-readable domain names, such as example.com, into IP addresses that computers use to locate servers.

DNS is an essential part of accessing websites because users can remember domain names more easily than IP addresses. DNS caching improves performance by reducing repeated lookups. However, DNS does not transfer the webpage itself; it helps the browser locate the destination server.

5.4 Suggest optimizations to improve performance.

The following techniques can improve browser-server communication:

1. Use reliable DNS services and appropriate DNS caching.
2. Enable HTTP/2 or HTTP/3 where supported.
3. Use HTTPS with efficient connection management.
4. Optimize server-side code and database queries.
5. Use browser caching and server-side caching.
6. Use a Content Delivery Network (CDN) to deliver resources closer to users.
7. Compress text-based resources using Brotli or Gzip.
8. Optimize images and use modern formats such as WebP or AVIF.
9. Minimize unnecessary JavaScript and CSS.
10. Lazy-load images and other non-critical resources where appropriate.
11. Monitor website performance using browser developer tools and performance metrics.

In conclusion, efficient DNS resolution, optimized server processing, caching, compression, and efficient browser rendering can reduce page loading time and improve the user experience.